

PROJECT REPORT

on

IoT Based Weather Monitoring System

Raspberry Pi Pico 2W | DHT11 | BMP280 | MQ-135 | Rain Sensor | TMP117 | ThingSpeak | Telegram Bot

DHARMSINH DESAI UNIVERSITY

FACULTY OF TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION

Guided by:

Prof. Sagar Kavaiya

Prepared by:

Yash Soni (23ECUOS105 – EC130)

Kaushik Paghdar (23ECUOS099 – EC126)

Academic Year: 2025 – 26

Certificate

This is to certify that the project entitled

“IoT Based Weather Monitoring System”

is the bonafide work of Yash Soni (EC130) and Kaushik Paghdar (EC126), students of B.Tech. Semester VI, Branch of Electronics and Communication Engineering, Dharmsinh Desai University. This project was carried out in the subject of Internet of Things (IoT) Practical Laboratory during the academic year 2025–26, under our guidance and supervision.

The content presented in this report is original and has not been submitted elsewhere for any other degree or examination.

Faculty In-charge:

Head of Department:

Prof. Sagar Kavaiya

Date: _____

Date: _____

Acknowledgement

We want to genuinely thank everyone who helped us in completing this project. It was not easy at the start since we had to manage multiple sensors, two microcontrollers, and cloud platforms all at once, so the support we received really made a difference.

First of all, a big thank you to Prof. Sagar Kavaiya for guiding us throughout this project. Whenever we got stuck, whether it was the BMP280 not giving proper readings or the Telegram Bot not sending messages correctly, he helped us figure out what was going wrong. His suggestions were always practical and to the point, which saved us a lot of time.

We also thank the Head of the Department and the lab staff for letting us use the electronics lab and the components we needed. The lab environment made it much easier for us to test and debug the circuits properly.

Thanks also to our friends and classmates who gave feedback when we were testing and helped us spot a few silly wiring mistakes that we had missed.

And of course, our families deserve a mention for being patient and supportive during the whole process, especially during the late nights before the submission.

Yash Soni & Kaushik Paghdar
B.Tech. Semester VI, EC

Index

Sr. No.	Title	Page No.
1	Certificate	2
2	Acknowledgement	3
3	Abstract	5
4	Introduction	6
5	Block Diagram	7
6	Block Diagram Explanation	8
7	Circuit Diagram	10
8	Components and Bill of Materials	11
9	Working Principle	14
10	Applications	17
11	Future Scope	18
12	Conclusion	19

Abstract

In this project we have built an IoT based weather monitoring system that can measure five different environmental parameters and send the data to both a cloud platform and a mobile phone in real time. We used two Raspberry Pi Pico 2W boards for this. The first one is connected to all the sensors, which include a DHT11 for temperature and humidity, a BMP280 for air pressure, an MQ-135 for air quality, and a rain drop sensor. It reads all these sensor values and sends them to ThingSpeak over Wi-Fi.

The second Pico 2W picks up that data from ThingSpeak and displays it on a 16x2 LCD screen. It also sends the readings to the user on their phone through a Telegram Bot. There is also a TMP117 precision temperature sensor connected to the second board so we can compare its reading with the DHT11 data from the first board. The whole system works without any manual input once it is set up and powered on. This project gave us a good understanding of how multi-sensor IoT systems are actually built and how cloud platforms like ThingSpeak work in practice.

Introduction

The idea behind this project was simple: we wanted to build something that actually works in real life, not just on paper. Weather monitoring is something that has practical use everywhere, from farming to factories to just knowing if it is going to rain before you step out. So we decided to make a system that reads multiple weather parameters, shows them locally on a display, and also sends them to the cloud so anyone can check them remotely.

We chose the Raspberry Pi Pico 2W because it has built-in Wi-Fi, which means we do not need any extra module for internet connectivity. We also liked that it can be programmed in MicroPython, which is easier to understand compared to writing code in C. The decision to use two Pico 2W boards came from the fact that we wanted to separate the sensing job from the display and notification job. This way, both tasks run independently and if one board has an issue, the other is not affected.

The sensors we picked cover the most important weather parameters. DHT11 gives temperature and humidity. BMP280 adds barometric pressure which is useful for predicting weather changes. MQ-135 measures air quality and can detect gases like CO₂, ammonia, and smoke. The rain sensor tells us if it is actually raining. And the TMP117 on the second board gives us a very accurate temperature reading to cross-check against the DHT11.

For the output side, we used a 16x2 LCD for local display and a Telegram Bot for mobile notifications. ThingSpeak acts as the middle layer that connects the two Pico 2W boards through the cloud without needing any direct wired connection between them. Overall this project covers sensors, I2C, ADC, Wi-Fi, HTTP APIs, JSON parsing, and mobile notifications, which together give a very practical and complete picture of how IoT systems work.

Block Diagram

The block diagram below shows how all the components are connected in our system. The left side shows the first Pico 2W which handles all the sensors. The right side shows the second Pico 2W which handles the display and Telegram notifications. ThingSpeak in the middle acts as the cloud bridge between the two boards.

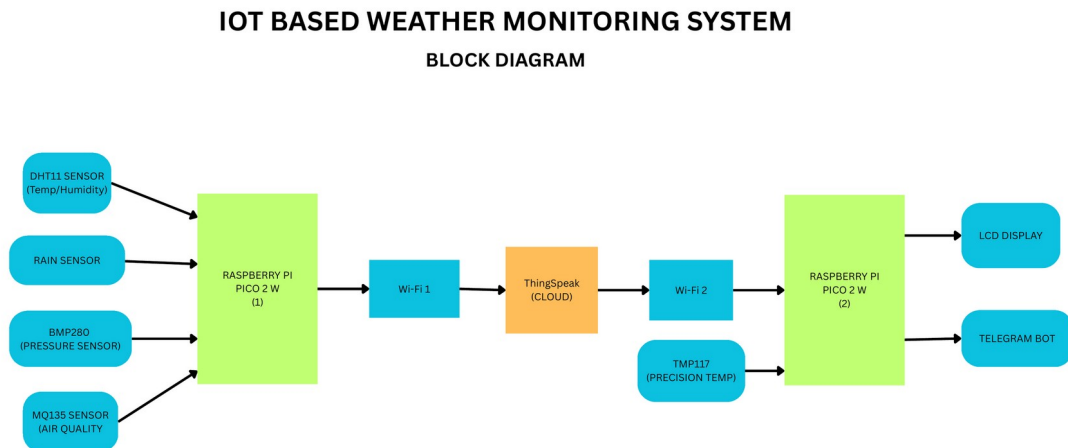


Figure 1: Block Diagram of IoT Based Weather Monitoring System

As shown in the diagram, four sensors feed data into Pico 2W (1): the DHT11, Rain Sensor, BMP280, and MQ-135. This board uploads all readings to ThingSpeak via Wi-Fi 1. Pico 2W (2) fetches the data from ThingSpeak via Wi-Fi 2. The TMP117 precision sensor is also connected directly to this second board. The outputs from Pico 2W (2) are the LCD Display and the Telegram Bot on the user's phone.

Block Diagram Explanation

Pico 2W (1) — Sensor Node

The first Pico 2W is only responsible for reading sensors and uploading data. We connected four sensors to it. The DHT11 uses a single GPIO pin and a special one-wire protocol to send temperature and humidity data. The BMP280 uses I2C, so it shares the SDA and SCL lines with the Pico. The MQ-135 gives an analog output which we read using the ADC pin on the Pico 2W. The rain sensor simply gives a HIGH or LOW digital output depending on whether water is on the probe board.

Once all four readings are taken, the board connects to Wi-Fi and sends an HTTP GET request to ThingSpeak with all the values as parameters in the URL. ThingSpeak stores this data with a timestamp in its cloud database. The board then waits for around 15 to 20 seconds before taking the next reading. We kept this gap because ThingSpeak free accounts require a minimum of 15 seconds between updates.

ThingSpeak Cloud

ThingSpeak is the platform that connects our two boards. The first board writes data to it and the second board reads from it. We created a channel on ThingSpeak with five fields: temperature, humidity, pressure, air quality, and rain status. Each field gets updated every cycle by Pico 2W (1). ThingSpeak also gives us time-series graphs on its dashboard which is useful for seeing trends over time.

Pico 2W (2) — Display and Notification Node

The second Pico 2W reads the latest data from ThingSpeak using the Read API and then does two things with it. First, it writes the values to the 16x2 LCD via I2C so that anyone near the device can see the current readings without needing a phone or computer. Second, it sends the readings as a message to a Telegram Bot so the user gets a notification on their phone. The TMP117 sensor is also read by this board directly over I2C, and its high-accuracy temperature reading is included in the Telegram message alongside the cloud data.

LCD Display

The 16x2 LCD with an I2C backpack is used for local real-time display. The I2C backpack reduces the number of wires needed from around 10 down to just 4 which made the wiring a lot cleaner on the breadboard. The display shows temperature, humidity, pressure, AQI, and rain status by cycling through them.

Telegram Bot

The Telegram Bot is one of the more interesting parts of this project. We created a bot using BotFather on Telegram and got an API token. The second Pico 2W sends an HTTP POST request to the Telegram API with the weather data formatted as a readable message. The user receives this message on their phone just like a regular Telegram chat message. This made the system feel genuinely useful because you can check the weather from anywhere just by opening Telegram.

Circuit Diagram

The circuit diagram shows the complete wiring of both Pico 2W boards along with all sensors, the LCD display, and power connections. Please insert your actual circuit photo or hand-drawn schematic in the space below.

[Insert Circuit Diagram / Practical Circuit Photo Here]

Figure 2: Circuit Diagram of IoT Based Weather Monitoring System

Pin Connections — Pico 2W Node 1 (Sensor Node)

Component	Component Pin	Pico 2W (1) Pin	Notes
DHT11	VCC	3.3V (Pin 36)	Power
DHT11	DATA	GP15 (Pin 20)	Single-wire data
DHT11	GND	GND (Pin 38)	Ground
BMP280	VCC	3.3V (Pin 36)	Power
BMP280	SDA	GP0 / I2C0 SDA	I2C Data
BMP280	SCL	GP1 / I2C0 SCL	I2C Clock
BMP280	GND	GND	Ground
MQ-135	VCC	VBUS 5V (Pin 40)	Needs 5V for heater
MQ-135	AOUT	GP26 / ADC0	Analog voltage output
MQ-135	GND	GND	Ground
Rain Sensor	VCC	3.3V	Power
Rain Sensor	DO	GP14 (Pin 19)	Digital output
Rain Sensor	GND	GND	Ground

Pin Connections — Pico 2W Node 2 (Display & Notification Node)

Component	Component Pin	Pico 2W (2) Pin	Notes
16x2 LCD (I2C)	VCC	VBUS 5V (Pin 40)	LCD needs 5V

16x2 LCD (I2C)	GND	GND	Ground
16x2 LCD (I2C)	SDA	GP0 / I2C0 SDA	I2C Data
16x2 LCD (I2C)	SCL	GP1 / I2C0 SCL	I2C Clock
TMP117	VCC	3.3V	Power
TMP117	SDA	GP0 / I2C0 SDA	Shared I2C bus
TMP117	SCL	GP1 / I2C0 SCL	Shared I2C bus
TMP117	GND	GND	Ground

Components and Bill of Materials

1. Raspberry Pi Pico 2W (x2)

The Raspberry Pi Pico 2W is the main controller we used in this project. It has a dual-core ARM Cortex-M33 processor, 264 KB RAM, and 4 MB flash memory. What made us choose this board is that it has built-in Wi-Fi through an Infineon CYW43439 chip, so we did not need any separate Wi-Fi module. It also supports MicroPython which made coding much easier. We bought two of these and used one as the sensor node and one as the display and notification node.

2. DHT11 Sensor

The DHT11 is a basic digital sensor that measures both temperature and humidity. It talks to the microcontroller through a single data wire using a custom protocol. The accuracy is not the best, around plus or minus 2 degrees Celsius for temperature and plus or minus 5 percent for humidity, but for a weather monitoring prototype it works fine. It is very cheap and easy to use with MicroPython since there is a built-in library for it.

3. BMP280 Pressure Sensor

The BMP280 measures atmospheric pressure and temperature. We mainly used it for the pressure reading, which is useful for detecting weather changes. When pressure drops suddenly, it usually means rain or a storm is coming. It communicates over I2C which made it easy to connect alongside other I2C devices on the same bus. It is accurate to about 0.01 hPa for pressure which is quite good for this kind of project.

4. MQ-135 Air Quality Sensor

The MQ-135 is a gas sensor that can detect CO₂, ammonia, alcohol, benzene, and smoke. It outputs an analog voltage that changes depending on the gas concentration in the air. We read this voltage using the ADC on the Pico 2W and convert it to an approximate PPM value. One thing to note is that this sensor needs 5V for its internal heater, so we powered it from the VBUS pin on the Pico. Also it needs a warm-up time of about a minute before readings become stable.

5. Rain Drop Sensor Module

This module has two parts: a probe board with copper traces and a comparator circuit board. When rain falls on the probe board, it reduces the resistance between the traces, and the comparator switches its digital output from HIGH to LOW. We connected this digital output to a GPIO pin on the first Pico 2W. There is a small potentiometer on the module to adjust how sensitive it is to moisture.

6. TMP117 Precision Temperature Sensor

The TMP117 is a high-accuracy digital temperature sensor from Texas Instruments. It is accurate to plus or minus 0.1 degree Celsius which is much better than the DHT11. We added this to the second Pico 2W node so we could compare its readings with the DHT11 data coming from the first node through ThingSpeak. It also uses I2C so it shares the same bus as the LCD on that board.

7. 16x2 LCD with I2C Backpack

The 16x2 LCD can display two rows of text with 16 characters each. On its own it needs around 10 wires to connect, but the I2C backpack module reduces that to just 4 wires. The backpack uses a PCF8574 chip to convert I2C commands into the parallel signals the LCD needs. We used this on the second Pico 2W to show the sensor readings locally on the device.

8. ThingSpeak Cloud Platform

ThingSpeak is a free IoT analytics platform where you can create channels and push data using simple HTTP requests. We created a channel with 5 fields for our sensor data. The first Pico 2W sends data to it and the second Pico 2W reads from it. It also shows graphs of each field which is helpful for checking trends over time. The free tier allows an update every 15 seconds which was enough for our project.

9. Telegram Bot

We made a Telegram Bot using BotFather and used its API to send weather messages from the second Pico 2W. The board sends an HTTP POST request to the Telegram API with the latest sensor data as the message text. This means the user gets a Telegram notification with all the weather parameters without needing to open any website.

Bill of Materials

Sr .	Component	Qty	Cost (Rs.)	Purpose
1	Raspberry Pi Pico 2W	2	1,180	Main microcontroller + Wi-Fi (x2)
2	DHT11 Sensor	1	80	Temperature & Humidity
3	BMP280 Module	1	120	Atmospheric Pressure
4	MQ-135 Gas Sensor	1	150	Air Quality / Gas Detection
5	Rain Drop Sensor Module	1	60	Rain Detection
6	TMP117 Precision Temp Sensor	1	350	High-accuracy Temperature
7	16x2 LCD Display (Blue)	1	90	Local Display
8	LCD I2C Interface Adapter	2	80	I2C adapter for LCD
9	Breadboard + Jumper Wires	1 set	80	Circuit assembly
10	USB Micro-B Cable	1	40	Power & Programming
11	5V Power Bank / Adapter	1	0	Power supply (own)
Total Estimated Cost			Approx. Rs. 2,230	

Working Principle

The whole system works in a loop. Node 1 keeps reading sensors and uploading to the cloud. Node 2 keeps fetching from the cloud and sending to the display and Telegram. Neither board needs to know what the other is doing directly, they just both talk to ThingSpeak independently.

Step 1: Startup

When we power on both boards, their MicroPython scripts start automatically. Node 1 sets up all the sensor pins and tries to connect to the Wi-Fi network using the SSID and password we hardcoded in the script. Node 2 initialises the LCD, shows a welcome message, connects to Wi-Fi the same way, and then waits a few seconds before starting the main loop. Both boards connect to the same Wi-Fi network.

Step 2: Reading Sensors on Node 1

Node 1 reads all four sensors one by one. For the DHT11, it pulls the data pin LOW for about 18 milliseconds to start a reading, then the sensor responds with 40 bits of data containing temperature and humidity. The MicroPython dht library handles all this automatically. For the BMP280, the board sends I2C requests to the sensor's registers and gets back raw ADC values which are then converted using calibration formulas from the datasheet. The MQ-135 analog output is read through the ADC and compared against a lookup to estimate PPM. The rain sensor is simply a digitalRead on the GPIO pin.

Step 3: Uploading to ThingSpeak from Node 1

After getting all the readings, Node 1 builds a URL like: `api.thingspeak.com/update?api_key=WRITEKEY&field1=28.5&field2=65&field3=1013&field4=400&field5=0` and sends it using the urequests library. If ThingSpeak responds with a channel entry number, the upload worked. If it fails, the board skips that cycle and tries again next time. After a successful upload, the board waits 15 seconds before the next reading.

Step 4: Fetching Data on Node 2

Node 2 calls the ThingSpeak Read API to get the last uploaded entry as a JSON response. It parses the JSON to extract each field value. Then it also reads the TMP117 directly via I2C. With all this data ready, it formats the values and starts updating the outputs.

Step 5: Displaying on LCD

The LCD can only show 16 characters per row. So we split the display into multiple screens that rotate every few seconds. For example, one screen shows temperature and humidity, the next shows pressure and AQI, and the next shows rain status and TMP117 temperature. The I2C backpack on the LCD handles the actual signal conversion, and we used the machine.I2C class in MicroPython to write to it.

Step 6: Sending Telegram Message

Node 2 sends an HTTP POST request to `https://api.telegram.org/bot{TOKEN}/sendMessage` with the chat ID and a message text. The message looks something like: Temp: 28.5C, Humidity: 65%, Pressure: 1013 hPa, AQI: 400 PPM, Rain: No, TMP117: 28.1C. The user receives this on their phone as a Telegram notification. We set it to send a new message every minute so the user gets regular updates without being flooded.

Step 7: Error Handling and Loop

We wrapped all sensor reads and HTTP calls in try-except blocks. If a DHT11 read fails due to a timing glitch, the previous value is shown instead. If the Wi-Fi drops, the board tries to reconnect automatically before the next cycle. This keeps the system running smoothly without needing someone to manually restart it.

Applications

Even though we built this as a college project, the system we made has several real-world uses. Here are some areas where this kind of setup can actually be deployed:

1. Agriculture and Farming

Farmers can place the sensor node in their fields to monitor temperature, humidity, pressure and rain. With the Telegram Bot, they can get weather alerts on their phone from anywhere on the farm. The rain detection helps them avoid unnecessary irrigation and the pressure reading gives an early warning of incoming storms.

2. Smart Home Monitoring

This system can be installed at home to keep track of indoor air quality using the MQ-135. If CO₂ or smoke levels go up, the Telegram Bot can alert the household. Combined with temperature and humidity, it gives a good overall picture of indoor comfort.

3. Industrial Safety

In industries that handle chemicals or gases, the MQ-135 can detect leaks of ammonia or other volatile compounds. Since the Telegram alert reaches the safety team immediately, they can respond faster than if they were relying on local alarms alone.

4. Educational Experiments

This project is a great practical example for any student learning IoT. It covers sensor interfacing, I2C, ADC, HTTP APIs, JSON, cloud platforms, and mobile notifications all in one system. Any college lab can replicate this setup at a reasonable cost.

5. Weather Research and Data Collection

If multiple units of this system are placed across a large area, they can form a distributed weather monitoring network. The data collected on ThingSpeak can be exported and analysed to study local climate patterns, air quality trends, or rainfall distribution.

6. Flood Warning Systems

The combination of rain detection and pressure drop monitoring makes this system useful as a basic flood warning tool. A sudden pressure drop combined with continuous rain detection can trigger an immediate alert to people in low-lying areas.

Future Scope

There are quite a few things we would like to add to this project if we get more time:

- The DHT11 is not very accurate. Replacing it with a DHT22 or SHT31 would give much better temperature and humidity readings, especially in outdoor environments where conditions change quickly.
- Adding a UV index sensor like the VEML6075 would be useful for outdoor health monitoring since high UV exposure is a serious concern in summer months.
- Right now the Telegram Bot only pushes messages. If we add a two-way interface, the user could type a command like /weather to the bot and get the latest readings on demand rather than waiting for the next scheduled update.
- Using MQTT instead of HTTP for ThingSpeak communication would reduce overhead and allow faster data updates, which would be important for time-critical monitoring scenarios.
- We would like to add a microSD card module for offline logging so no readings are lost even if the Wi-Fi goes down for a few hours. The stored data could be uploaded in batch when the connection is restored.
- Adding a solar panel and a LiPo battery management circuit would make the sensor node completely wireless and self-powered, which is important for outdoor field deployments.
- A simple ML model trained on the historical ThingSpeak data could eventually be used to predict rain or temperature trends for the next few hours, turning this into a basic forecasting tool.
- Adding a GPS module would allow the system to tag all readings with the exact location, which is useful if multiple nodes are deployed across a large area.

Conclusion

We set out to build a weather monitoring system that goes beyond just reading one or two sensors. What we ended up with is a system that reads five environmental parameters, stores them on the cloud, shows them on a local display, and notifies the user on their phone through Telegram. The dual-board architecture using two Raspberry Pi Pico 2W boards was a bit more challenging to set up but it made the system more organised and closer to how real IoT systems are actually designed.

Each part of the project taught us something different. Getting the BMP280 and TMP117 working on the same I2C bus required us to understand I2C addressing properly. The MQ-135 needed careful ADC calibration. Setting up the Telegram Bot and making the Pico 2W send HTTP POST requests was something we had not done before, and getting that to work was quite satisfying. ThingSpeak also turned out to be more useful than we expected, especially the graphing feature which makes it easy to visualise trends over time.

Overall this project gave us a solid hands-on understanding of IoT systems. We now feel much more confident about working with sensors, communication protocols, cloud APIs, and microcontrollers together as a system rather than each in isolation. The ideas listed in the future scope section are things we genuinely want to pursue, and we think this project has a strong base to build on further.